Clase is an object-oriented programming language designed to make AI coding easier by mimicking the natural flow of handwriting. Its defining characteristic is a strict left-to-right flow encompassing assignments, variable declarations, function calls, and input/output operations. The language draws foundational inspiration from C++ for variable declarations and stream I/O, and Common Lisp for its explicit, self-closing semitagged blocks.

Lexical & Syntax Fundamentals

- **Assignment Operator:** The traditional assignment flow is reversed, using > instead of = (e.g., a + b > a). The > symbol retains its standard meaning as "greater than" when used within conditionals.
- **Semitagged Blocks:** Control flow and structural keywords operate as explicit, self-closing blocks enclosed in pipe | characters, resembling a markup language.
- **Comments:** Single-line comments are denoted by //.
- **Memory Management:** The language utilizes a built-in garbage collector.
- **Source Files:** Module files use the .Clase extension.

Data Types, Vectors & Variables

Clase includes native support for standard data types and specifically utilizes vectors over arrays for improved efficiency.

- **Primitive Types:** int (integer), float (decimal), char (character), bool (boolean true/false), and string (a vector of char).
- **Operators:** Supports standard arithmetic operations: +, -, *, /, and %.
- **Variable Declaration:** Variables flow left-to-right and can be declared independently, grouped by type separated by whitespace, or initialized inline using parentheses (e.g., int i(0), int a b).
- **Vectors:** Vector syntax replaces brackets [ ] with pipes | |.
    - Declaration: type name|size|.
    - Initialization: type name|size|(|val1, val2|).
    - Access: name|index| using 0-based indexing.

Program Structure & Control Flow

Module & Intermodular Definitions

Programs are encapsulated in module| ... |module blocks. Modules can access standard I/O and link to other .Clase files using a preceding intermodular| block containing link| filename.Clase |link declarations.

Functions

- **Declaration:** fun| <parameters> > <function_name> > <return_type>. Parameters are space-separated.
- **Return Statements:** Values are returned using back| expression |back.
- **Execution Call:** Arguments are passed left-to-right (e.g., arg1 arg2 > function_name).
- **Main Entry:** Programs begin at fun| string args| | > main > int.
- **I/O Chaining:** Input, function calls, and output can be seamlessly combined in a single left-to-right sequence (e.g., in > a b > Euclid > out).

Keyword Substitutions & Loops

Clase renames standard programming control structures:

- **If / Else:** Replaced by when| ... other| ... |other |when.
- **While Loop:** Replaced by repeat| condition ... |repeat.
- **For Loop:** Replaced by iterate|. The loop signature tracks the iterator, initialization, step, and limit (e.g., iterate| int i(0)++ < n ... |iterate).

Advanced Paradigms

Object-Oriented Programming (OOP)

- **Classes:** Declared using class| ClassName ... |class.
- **Members:** Can contain both data members and internal fun| member functions.
- **Instantiation & Access:** Objects are created via ClassName ObjectName and members are accessed using standard dot notation (e.g., Object.member).

Metaprogramming & Compile-Time Execution

- **Compilation-Time Blocks:** Preprocessors and templates are replaced by comp| ... |comp blocks, allowing standard code evaluation at build time. Compile-time classes are supported.
- **Polymorphism & Generics:** Generics bypass complex template rules via procedural specialization, using explicit variable bounds &T (for types) and &Op (for operators).

Concurrency & File System

- **Concurrency:** Handled via Multitex tasks. Tasks are stored in a vector and executed simultaneously using the .concurr method (e.g., tasks|i|.concurr).
- **File I/O:** Declared with file f(name, type, options) where type is bin, txt, or hex, and options are r or w. File operations include .open, .close, and .eof, with read/write handled via left-to-right flow (s > f to write, f > s to read).

Abstract Syntax Tree (AST) Specification

The following represents the hierarchical AST structure derived from the structural rules of the Clase language.

- **ProgramNode**
  - IntermodularNode (Optional)
    - List of LinkNode: [String: filename]
  - ModuleNode
    - String: Identifier
    - List of DeclarationNodes (Functions, Classes, CompBlocks)
- **DeclarationNodes**
  - **ClassNode**
    - String: ClassName
    - List of VariableDeclarationNode
    - List of FunctionNode
  - **FunctionNode**
    - List of ParameterNode (Type, Identifier)
    - String: FunctionName
    - TypeNode: ReturnType
    - BlockNode: Body
  - **CompBlockNode** (Compile-time wrapper)
    - List of DeclarationNodes
- **StatementNodes**
  - **VariableDeclarationNode**
    - TypeNode: Type
    - List of VariableInitializerNode: [String: Identifier, ExpressionNode: InitialValue (Optional)]
  - **VectorDeclarationNode**
    - TypeNode: Type
    - String: Identifier

        - ExpressionNode: Size

        - List of ExpressionNode: Initializers (Optional)
    - **AssignmentNode** (Left-to-right assignment)
        - ExpressionNode: Value (Left side)
        - IdentifierNode: TargetVariable (Right side)
    - **ReturnNode** (back|)
        - ExpressionNode: Value

    - **FileOperationNode**
        - IdentifierNode: FileObject

        - String: Method (.open, .close, etc.)
- **ControlFlowNodes**
    - **WhenNode** (when|)
        - ExpressionNode: Condition

        - BlockNode: TrueBody

        - OtherNode: FalseBody (Optional, maps to other|)
    - **RepeatNode** (repeat|)
        - ExpressionNode: Condition

        - BlockNode: Body

    - **IterateNode** (iterate|)
        - VariableDeclarationNode: Iterator

        - String: StepOperator (e.g., ++)
        - ExpressionNode: LimitCondition

        - BlockNode: Body

- **ExpressionNodes**
    - FunctionCallNode: [List of ExpressionNode: Arguments, String: FunctionName]

    - BinaryOperationNode: [ExpressionNode: Left, String: Operator, ExpressionNode: Right]
    - MemberAccessNode: [String: ObjectName, String: MemberName] (e.g., N.a)
    - VectorAccessNode: [String: VectorName, ExpressionNode: Index]

    - LiteralNode: (Integer, Float, Char, String, Bool)